

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

- Terminal showing your current branch

```
Problems Output Debug Console Terminal Ports
PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese\my_first_flutter_app> git branch
* Feature/widget
  main
PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese\my_first_flutter_app> git switch -c feature/data-ui
Switched to a new branch 'feature/data-ui'
PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese\my_first_flutter_app> git branch
  Feature/widget
  Feature/data-ui
  main
PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese\my_first_flutter_app> 
```

Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Create:

- Your identified data
- The corresponding data types

Name	String
age	int

Height	double
Student Status	bool



Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

Create variables for at least:

- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**. (Declare variables separately from your UI, not directly inside your widgets)

Screenshot:

- Your variables
- Their data types
- Application still running

```
Run | Debug | Profile
void main() => runApp(const MyApp());

//Profile Data Class
class Profile {
  final String name;
  final String course;
  final int age;
  final String hobby;
  final double height;
  final bool isStudent;
  final String image;

  const Profile({
    required this.name,
    required this.course,
    required this.age,
    required this.hobby,
    required this.height,
    required this.isStudent,
    required this.image,
  });
}

const profiles = [
```



Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

Test:

Change the value of one of your variables.

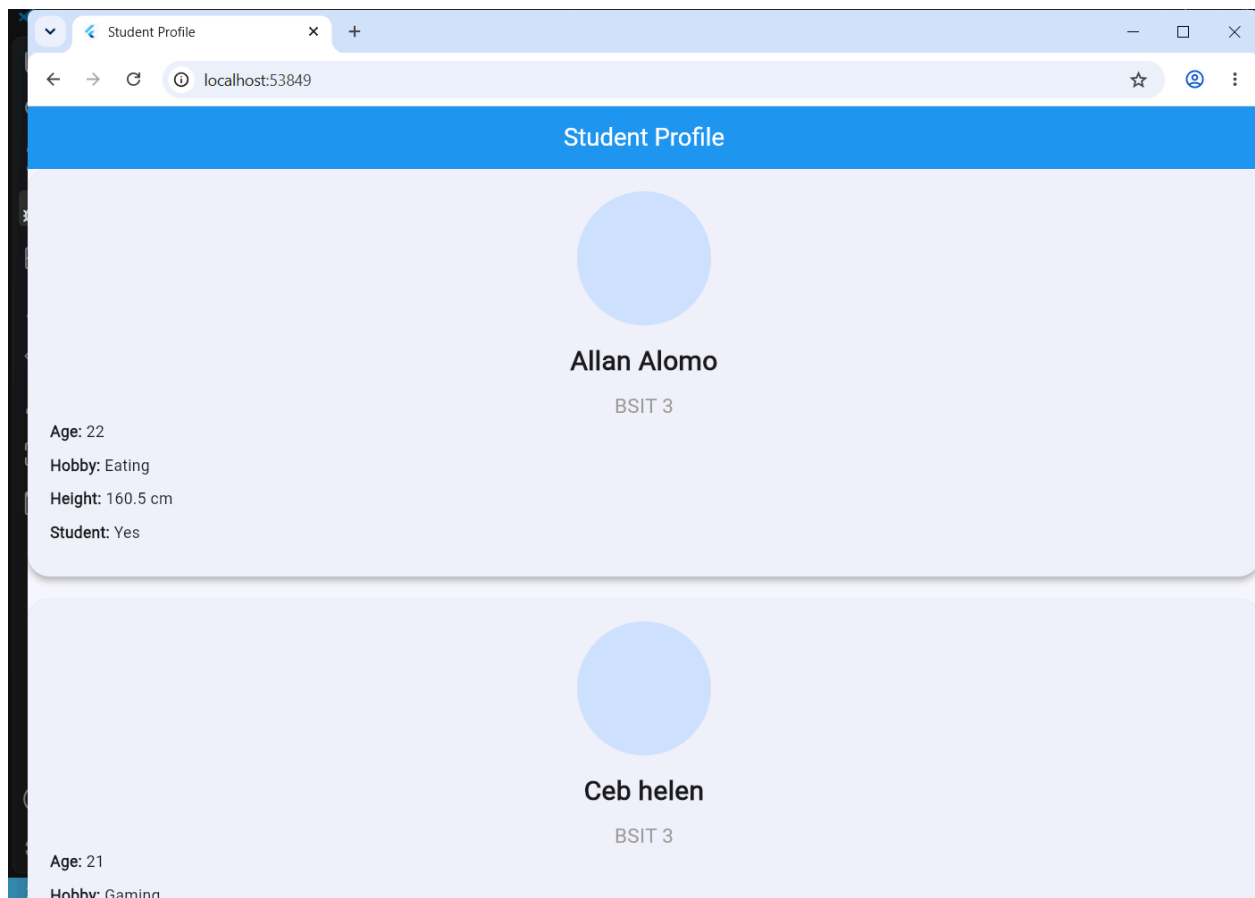
The UI should automatically display the new value.

Screenshot:

- Variables
- Widgets displaying the variables

```
const profiles = [
  Profile(
    name: 'Allan Alomo',
    course: 'BSIT 3',
    age: 22,
    hobby: 'Eating',
    height: 160.5,
    isStudent: true,
    image: 'assets/Allan.jfif',
  ),
  Profile(
```

- Updated application



🚩 Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

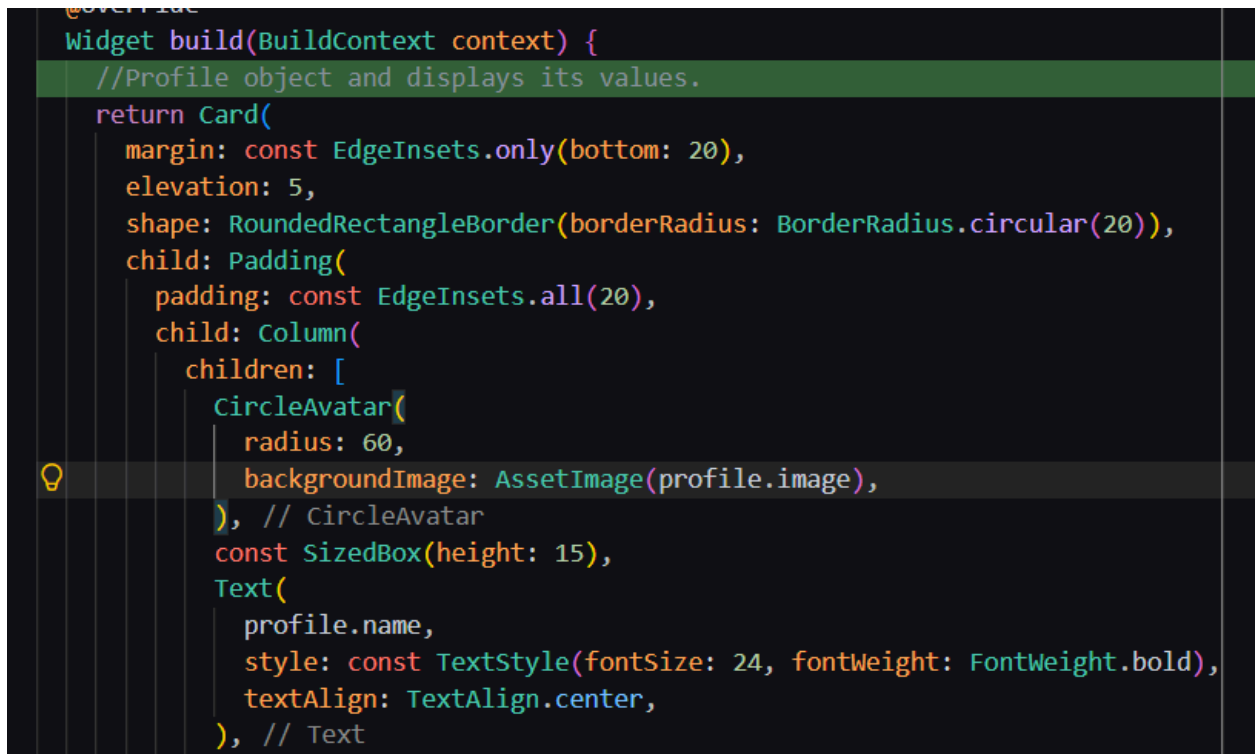
Use the variable when displaying the image.

Then change the variable to another registered image.

The image displayed by the application should change.

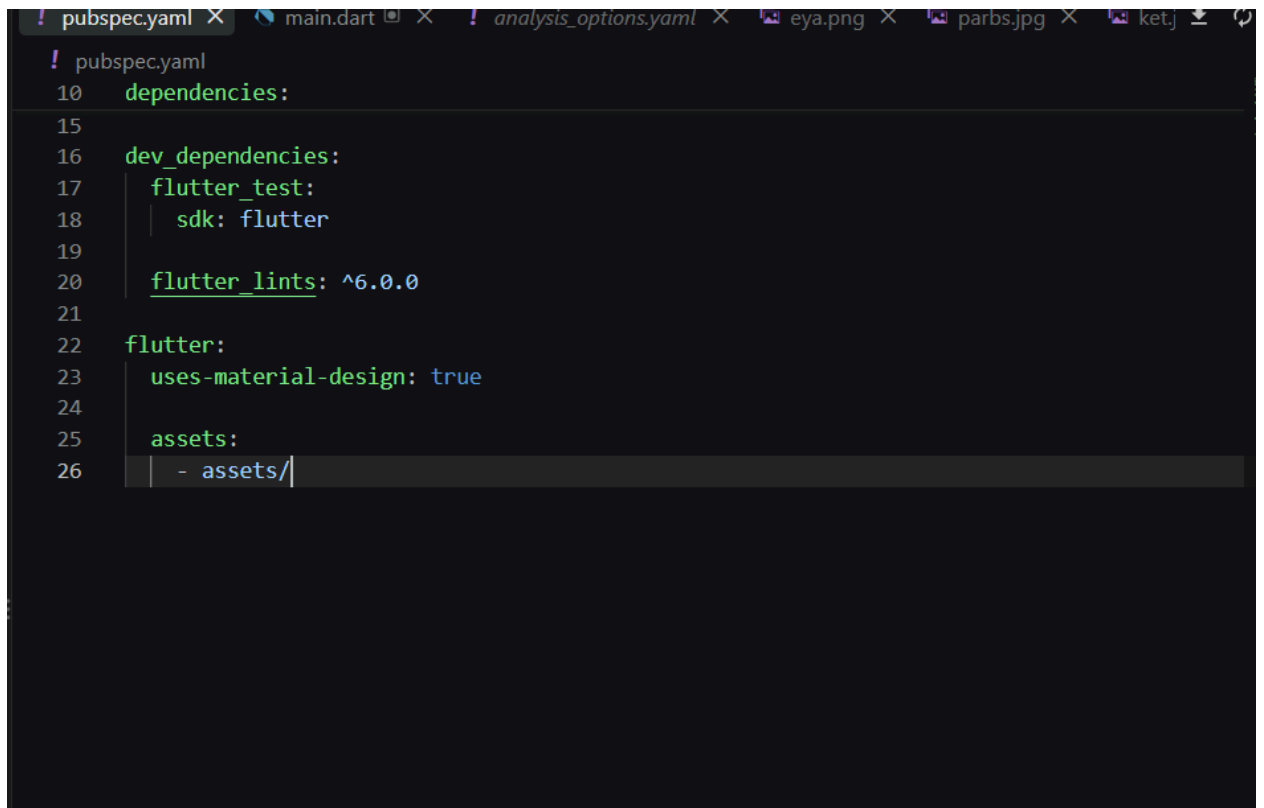
Screenshot:

- Image variable



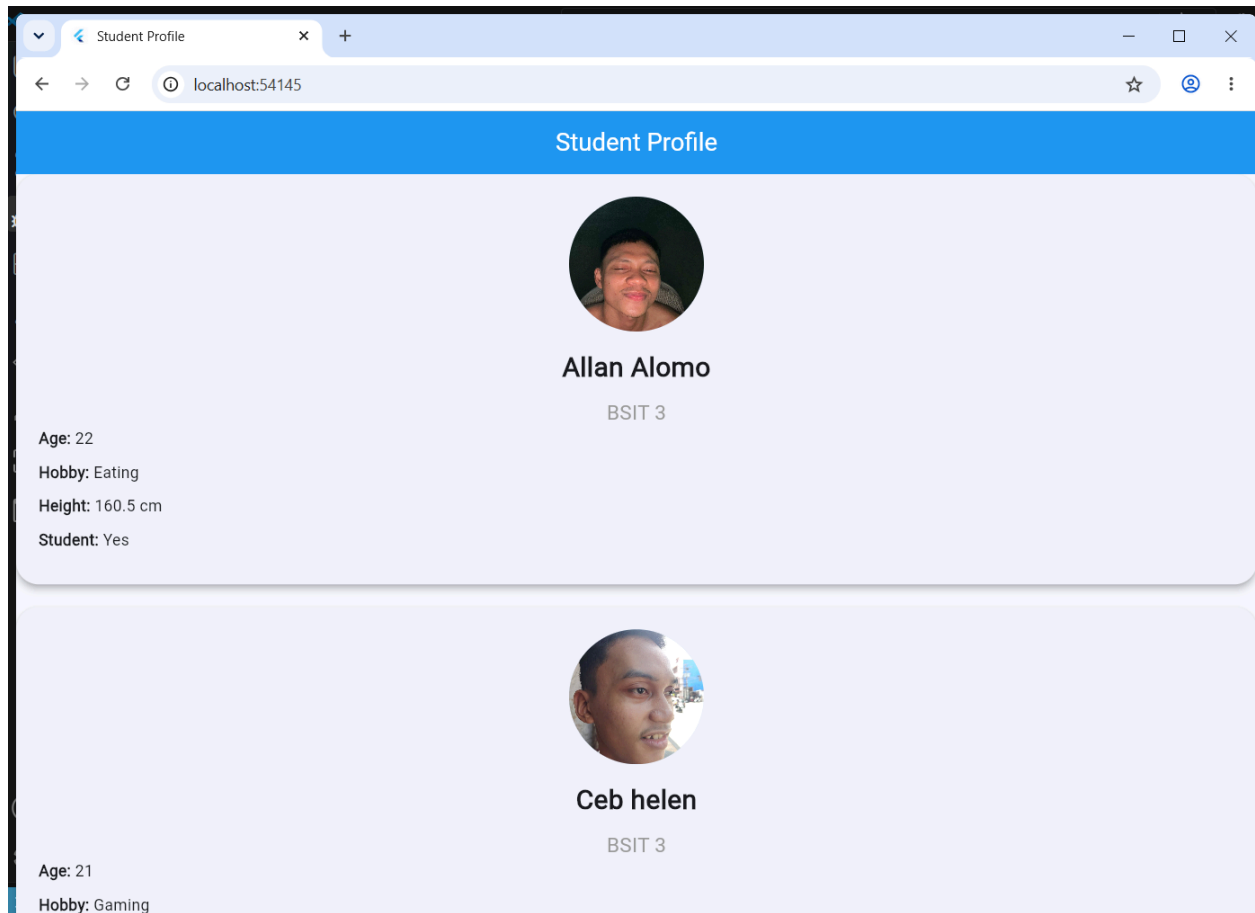
```
@override
Widget build(BuildContext context) {
  //Profile object and displays its values.
  return Card(
    margin: const EdgeInsets.only(bottom: 20),
    elevation: 5,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Column(
        children: [
          CircleAvatar(
            radius: 60,
            backgroundImage: AssetImage(profile.image),
          ), // CircleAvatar
          const SizedBox(height: 15),
          Text(
            profile.name,
            style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
            textAlign: TextAlign.center,
          ), // Text
        ],
      ),
    ),
  );
}
```

- Image widget using the variable



```
pubspec.yaml X main.dart X analysis_options.yaml X eya.png X parbs.jpg X ket.j
! pubspec.yaml
10 dependencies:
15
16 dev_dependencies:
17   flutter_test:
18     sdk: flutter
19
20   flutter_lints: ^6.0.0
21
22 flutter:
23   uses-material-design: true
24
25   assets:
26     - assets/
```

- Updated application



Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data

```
class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Student Profile'),
        centerTitle: true,
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ), // AppBar
      // Creates one ProfileCard for each item in profiles.
      body: ListView.builder(
        itemCount: profiles.length,
        itemBuilder: (_, index) => ProfileCard(profile: profiles[index]),
      ), // ListView.builder
    ); // Scaffold
  }
}
```

Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

The user must be able to:

Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:

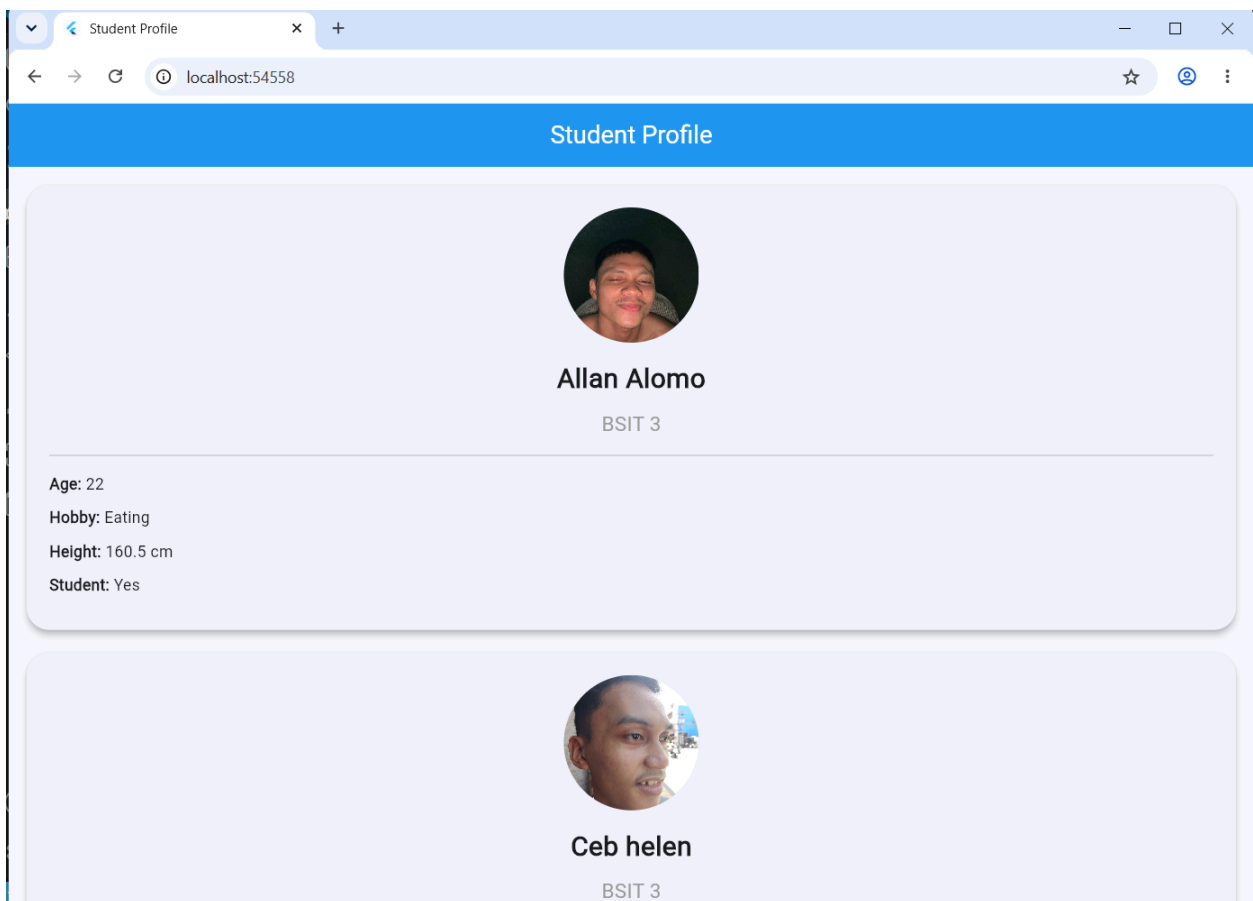
- The scrolling widget in your code

```

102         centerTitle: true,
103         backgroundColor: Colors.blue,
104         foregroundColor: Colors.white,
105     ), // AppBar
106     // Creates one ProfileCard for each item in profiles.
107     body: ListView.builder(
108         padding: const EdgeInsets.all(16),
109         itemCount: profiles.length,
110         itemBuilder: (_, index) => ProfileCard(profile: profiles[index]),
111     ), // ListView.builder
112 ); // Scaffold
113 }
114 }
115

```

- Top portion of the application
- The rest of the portion of the application showing the later profiles





Ceb helen

BSIT 3

Age: 21
Hobby: Gaming
Height: 170 cm
Student: Yes



Kenwel Novetnom

BSIT 3

Age: 19
Hobby: Reading
Height: 155 cm
Student: Yes

Student Profile

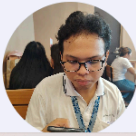
Height: 155 cm
Student: Yes

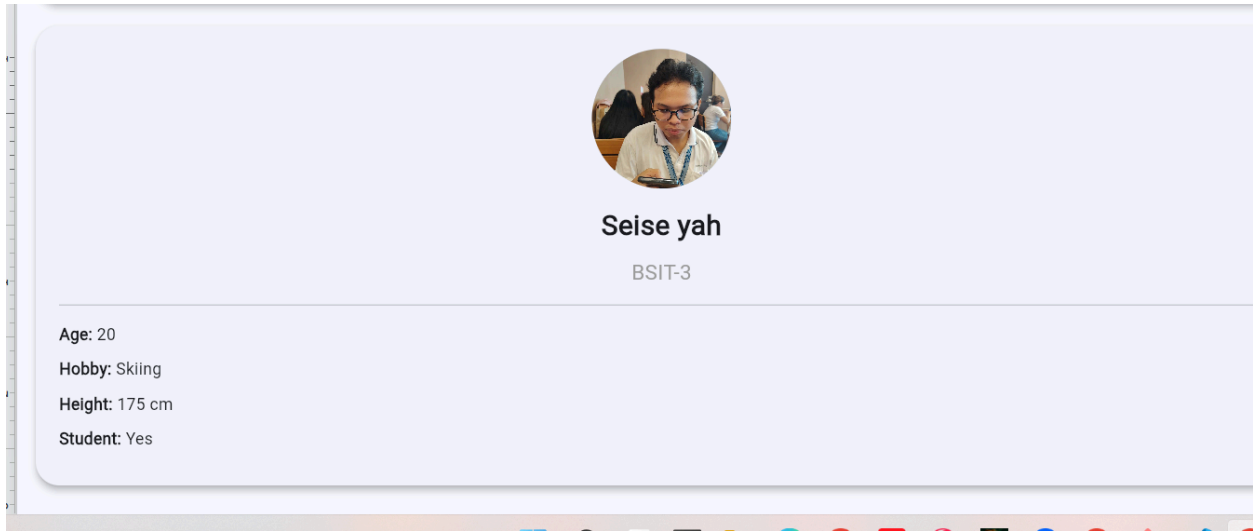


Keth Lopes

BSIT 3

Age: 22
Hobby: Computer Programming
Height: 180 cm
Student: Yes





🚩 Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: missing

Profile B

Name: Juan

Course: missing

Hobby: Gaming

Profile C

Name: missing

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

- Incomplete profile data

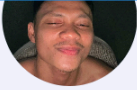
```

26  const profiles = [
27      Profile(
28          name: 'Allan Alomo',
29          course: 'BSIT 3',
30          age: 22,
31          hobby: 'Not Provided',
32          height: 160.5,
33          isStudent: true,
34          image: 'assets/Allan.jfif',
35      ),
36      Profile(
37          name: 'Ceb helen',
38          course: 'Unknown',
39          age: 21,
40          hobby: 'Gaming',
41          height: 170,
42          isStudent: true,
43          image: 'assets/parbs.jpg',
44      ),
45      Profile(
46          name: 'Kenwel Novetnom',
47          course: 'BSIT 3',
48          age: 19,
49          hobby: 'Reading',
50          height: 155,
51          isStudent: true,
52          image: 'assets/profile.jfif',
53      ),
54      Profile(
55          name: 'Keth Lopes',
56          course: 'BSIT 3',
57          age: 22,
58          hobby: 'Computer Programming',
59          height: 180,
60          isStudent: true,

```

- Application displaying fallback values

Student Profile



Allan Alomo

BSIT 3

Age: 22
Hobby: Not Provided
Height: 160.5 cm
Student: Yes

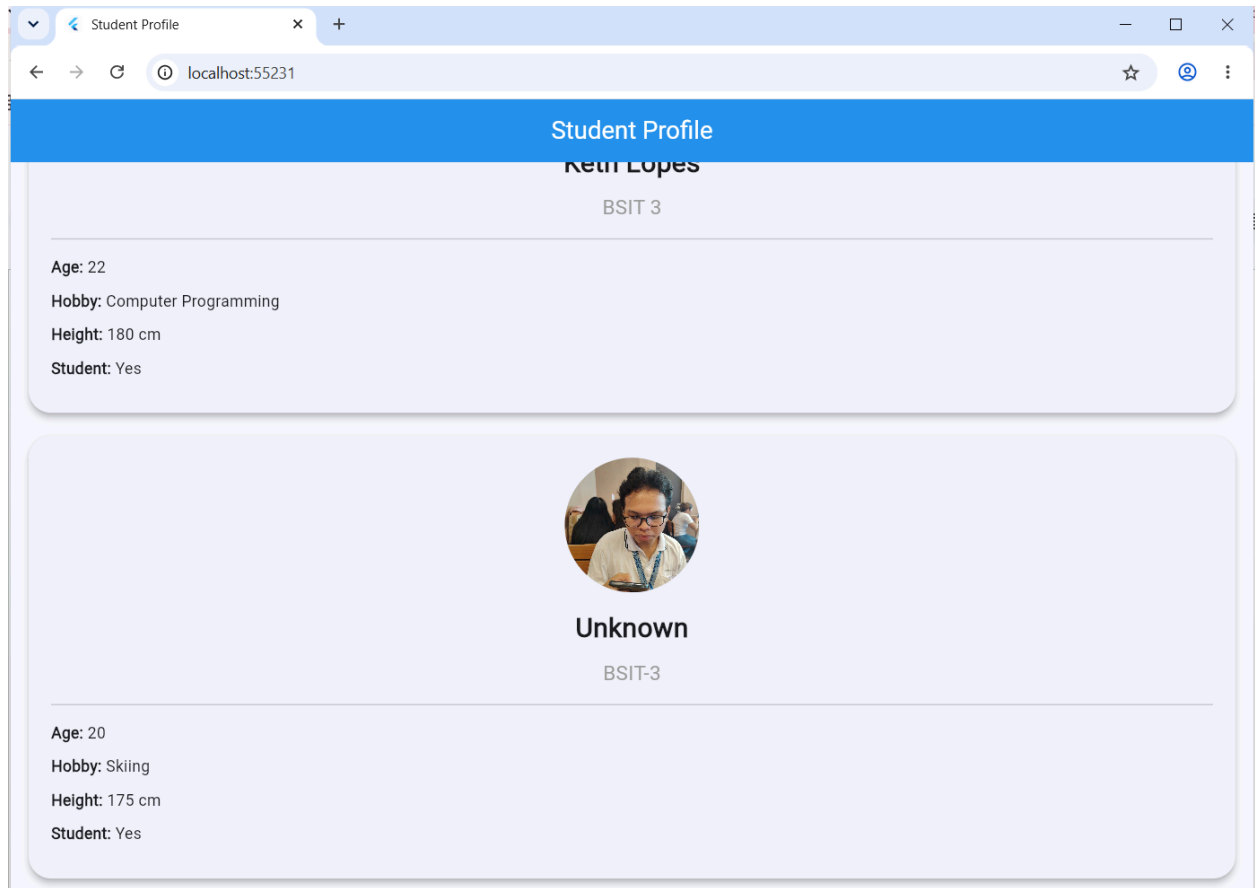


Ceb helen

Unknown

Age: 21
Hobby: Gaming

-
- Application still functioning correctly



Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

```
Windows PowerShell
(use "git restore <file>..." to discard changes in working directory)
deleted: CTF_2.pdf

PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese> dir

Directory: C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese

Mode                LastWriteTime         Length Name
----                -
d-----          9/17/2026   6:05 AM                my_first_flutter_app

PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese> git add .
PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese> git commit -m "feat: Complete Data Driven UI"
[Feature/data-ui 3059080] feat: Complete Data Driven UI
8 files changed, 157 insertions(+), 224 deletions(-)
delete mode 100644 CTF_2.pdf
create mode 100644 my_first_flutter_app/assets/Allan.jfif
create mode 100644 my_first_flutter_app/assets/Shibal.jfif
create mode 100644 my_first_flutter_app/assets/eya.png
create mode 100644 my_first_flutter_app/assets/ket.jpg
create mode 100644 my_first_flutter_app/assets/parbs.jpg
PS C:\Users\Ninja Turtle\Documents\binsoy\CTF_3\IT314AB_Rebese> git push origin Feature/data-ui
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (12/12), 2.09 MiB | 1.09 MiB/s, done.
```